

Informe Técnico Analítico

Grupo 46



Facultad de Ciencias Exactas y Naturales y Agrimensura.

Integrantes: Acosta, Lucas; Gauto, Gimena; Romero Alegre, Benjamin;
Pindo, Angel Francisco.

Fecha: 16/06/2026

Fundamentos Diseño del Código

Hemos optado por un estilo funcional centrado principalmente en la obtención de resultados a partir de funciones que reciben parámetros y devuelvan valores, sin llegar a modificar o alterar los parámetros utilizados. Para esto, evitamos el uso de variables globales innecesarias y recurrimos en su lugar a las variables locales, que actúan como auxiliares, para principalmente fijar un valor que será utilizado constantemente como el de la duración de un ciclo semafórico o en un cálculo matemático. Por otra parte, la modularización en ciertas funciones fue fundamental para dividir problemas en partes y tener más claro el panorama si se llegase a producir un error a la hora de ejecutar el código.

Elección de la librería “local-time”

La decisión de seleccionar a la librería local fue debido a que queríamos hacer entendible las salidas de los códigos que usaban tiempo Unix como parámetro. El mejor ejemplo de ello son las funciones de Auditoría que devuelven la fecha entendible para comprobar que el resultado sea el correcto.

Respuestas Teóricas Scala

- 1) Primeramente para generar la estructura del semáforo definimos un tipo de dato enum llamado Color, el cual iba a tener las opciones de justamente los colores e intermitencias. Nosotros elegimos agrupar todo dentro de un object debido a que este actúa como un contenedor de funciones puras, transicion y temporizador son funciones puras ya que solamente reciben el dato, lo calculan y devuelven el resultado sin modificar nada por fuera.
- 2) En Lisp las funciones de orden superior reciben la función como argumento al principio y la lista al final, en cambio, en scala se parte de la colección y se encadenan los métodos usando la

notación de punto (.) y el guión bajo (_) para funciones anónimas cortas.

Por ejemplo, lisp: (mapcar #'(lambda (x) (* x 2)) lista), scala: lista.map(_ * 2).

Sabiendo esto podemos decir que resulta más legible la escritura en scala porque el mismo se lee de izquierda a derecha en el orden real en el que suceden las cosas, a diferencia de lisp que hay que ir leyendo de adentro hacia afuera, además con el guión bajo(_), scala también elimina la necesidad de declarar variables a mano.

Bitácora de Depuración

1) Error de salida por pantalla.

Solución: Cambiar por Format Nil a Format T.

```
(defun transicion(color-Actual cambiar-a)
  (cond
    ((and (equal color-Actual 'en-Verde) (equal cambiar-a 'amarillo)) ;cambio valido
      (list color-Actual (format t "cambiar-a-~a" cambiar-a)))

    ((and (equal color-Actual 'en-Amarillo) (equal cambiar-a 'rojo)) ;cambio valido
      (list color-Actual (format t "cambiar-a-~a" cambiar-a)))

    ((and (equal color-Actual 'en-Rojo) (equal cambiar-a 'verde)) ;cambio valido
      (list color-Actual (format t "cambiar-a-~a" cambiar-a)))

    ((and (equal color-Actual 'en-Verde) (equal cambiar-a 'rojo)) ;cambio NO valido
      (list color-Actual "Cambio esperado: AMARILLO"))

    ((and (equal color-Actual 'en-Amarillo) (equal cambiar-a 'verde)) ;cambio NO valido
      (list color-Actual "Cambio esperado: ROJO"))

    ((and (equal color-Actual 'en-Rojo) (equal cambiar-a 'amarillo)) ;cambio NO valido
      (list color-Actual "Cambio esperado: VERDE")))

    (t (list "Datos no Validos")))
  )
)

(transicion 'en-Verde 'amarillo)

TRANSICION
[4]> cambiar-a-AMARILLO
(EN-VERDE NIL)
```

Explicación: Format t imprime en pantalla, lo que provoca que como se utiliza list lo último que devuelve

sea el nil. Mientras que format nil devuelve una cadena de texto.

2) Error al usar la librería “local-time”.

```
*** - READ from
      #<INPUT CONCATENATED-STREAM
      #<INPUT UNBUFFERED FILE-STREAM CHARACTER #P"/dev/fd/0" @14>>
      : there is no package with name "LOCAL-TIME"
The following restarts are available:
ABORT      :R1   Abort main loop
Break 1 [2]>
NIL
Break 1 [2]>
*** - READ from
      #<INPUT CONCATENATED-STREAM #<INPUT STRING-INPUT-STREAM>
      #<IO TWO-WAY-STREAM
      #<INPUT UNBUFFERED FILE-STREAM CHARACTER #P"/dev/fd/0" @16>
      #<OUTPUT UNBUFFERED FILE-STREAM CHARACTER #P"/dev/fd/1">>>
      : there is no package with name "LOCAL-TIME"
The following restarts are available:
ABORT      :R1   Abort debug loop
ABORT      :R2   Abort main loop
Break 2 [3]>
:FORMAT
Break 2 [3]>
(:YEAR "-" :MONTH "-" :DAY " " :HOUR ":" :MIN ":" :SEC)
Break 2 [3]>
*** - READ from
      #<INPUT CONCATENATED-STREAM #<INPUT STRING-INPUT-STREAM>
      #<IO TWO-WAY-STREAM
      #<INPUT UNBUFFERED FILE-STREAM CHARACTER #P"/dev/fd/0" @17>
      #<OUTPUT UNBUFFERED FILE-STREAM CHARACTER #P"/dev/fd/1">>>
      : an object cannot start with #\
The following restarts are available:
ABORT      :R1   Abort debug loop
ABORT      :R2   Abort debug loop
ABORT      :R3   Abort main loop
Break 3 [4]>
*** - EVAL: undefined function CALCULAR-TIEMPO
The following restarts are available:
USE-VALUE  :R1   Input a value to be used instead of (FDEFINITION 'CALCULAR-TIEMPO).
RETRY      :R2   Retry
STORE-VALUE :R3   Input a new value for (FDEFINITION 'CALCULAR-TIEMPO).
ABORT      :R4   Abort debug loop
ABORT      :R5   Abort debug loop
ABORT      :R6   Abort debug loop
ABORT      :R7   Abort main loop
Break 4 [5]>
*** - READ from
      #<INPUT CONCATENATED-STREAM #<INPUT STRING-INPUT-STREAM>
      #<IO TWO-WAY-STREAM
      #<INPUT UNBUFFERED FILE-STREAM CHARACTER #P"/dev/fd/0" @19>
      #<OUTPUT UNBUFFERED FILE-STREAM CHARACTER #P"/dev/fd/1">>>
      : an object cannot start with #\
The following restarts are available:
ABORT      :R1   Abort debug loop
ABORT      :R2   Abort debug loop
ABORT      :R3   Abort debug loop
ABORT      :R4   Abort debug loop
ABORT      :R5   Abort main loop
Break 5 [6]>
*** - READ from
      #<INPUT CONCATENATED-STREAM #<INPUT STRING-INPUT-STREAM>
```

Solución: cargar (ql:quickload "local-time").

Explicación: Para utilizar cualquier tipo de librería, primero debe de inicializarse la misma.

3) Error no se guarda el historial de la Auditoría en la Extensión 2.

```
"Ingrese su ID: " Francisco
"Ingrese la clave: " 1234
"Ingrese tiempo Unix para saber el color anterior y el siguiente: " 1781621963
("Usuario: FRANCISCO|| Tiempo Unix ingresado: 1781621963" "2026-6-16 14:59:23"
 "La luz ha cambiado de Rojo a Verde")
[6]> (Auditoria)

"Ingrese su ID: " jorge
"Ingrese la clave: " 1234
"Ingrese tiempo Unix para saber el color anterior y el siguiente: " 1781651963
("Usuario: JORGE|| Tiempo Unix ingresado: 1781651963" "2026-6-16 23:19:23"
 "La luz ha cambiado de Amarillo a Rojo")
```

```
Informe de Ejecucion del Sistema Semaforico
2026-6-16 15:32:43 - [Usuario: FRANCISCO|| Tiempo Unix ingresado: 1781623963] --- Transición: La luz ha cambiado de Rojo a Verde
=====
```

Solución: Mantener abierto el .txt.

Explicación: Al escribirse sobre el mismo archivo

4)Error al cargar el archivo en sbcl.

The screenshot shows a Sublime Text editor on the left with a file named `core.lisp` containing the following code:

```
1 ;2
2
3
4 (defun timer(tiempo_Unix)
5
6   (let ((duracion_Ciclo 216)) ; tiempo del ciclo completo
7     (cond
8       ((and (>= (mod tiempo_Unix duracion_Ciclo) 0) (<= (mod
9         ((and (>= (mod tiempo_Unix duracion_Ciclo) 90) (<= (mo
10        ((and (>= (mod tiempo_Unix duracion_Ciclo) 210) (<= (m
11      )
12 )
13 )
```

On the right, a Windows PowerShell window shows the output of the `sbcl` command. It displays the SBCL version (2.6.5) and then an error message:

```
PS C:\Users\Estudiante\Desktop\2do año\Paradigmas y lenguajes> sbcl
This is SBCL 2.6.5, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>

SBCL is free software, provided as is, with absolutely no warranty
It is mostly in the public domain; some portions are provided unde
r
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (load "core.lisp")
While evaluating the form starting at line 4, column 0
of #P"C:/Users/Estudiante/Desktop/2do año/Paradigmas y lenguajes
/core.lisp":
debugger invoked on a SYMBOL-PACKAGE-LOCKED-ERROR in thread
#<THREAD tid=2900 "main thread" RUNNING {1104030083}>:
Lock on package SB-EXT violated when proclaiming TIMER as a func
tion while in
package COMMON-LISP-USER.
See also:
The SBCL Manual, Node "Package Locks"
Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
0: [CONTINUE] Ignore the package lock.
1: [IGNORE-ALL] Ignore all package locks in the context of t
his operation.
2: [UNLOCK-PACKAGE] Unlock the package.
3: [RETRY] Retry EVAL of current toplevel form.
4: Ignore error and continue loading file "C:\\
Users\\Estudiante\\Desktop\\2do año\\Paradigmas y lenguajes\\core.
lisp".
5: [ABORT] Abort loading file "C:\\Users\\Estudiante\\D
esktop\\2do año\\Paradigmas y lenguajes\\core.lisp".
6: Exit debugger, returning to top level.

(PACKAGE-LOCK-VIOLATION #<PACKAGE "SB-EXT"> :SYMBOL TIMER :FORMAT-
CONTROL "proclaiming ~S as a function" :FORMAT-ARGUMENTS (TIMER))
0]
```

Solución: modificar el nombre del requerimiento 2, timer a temporizador, debido que sbcl tiene definido internamente una función llamada justamente “timer” por lo cual nos da error al querer cargar un archivo con una función que tenga el mismo nombre.

The screenshot shows the same Sublime Text editor on the left, but the code in `core.lisp` has been modified to use the name `temporizador` instead of `timer`:

```
1 ;2
2
3
4 (defun temporizador(tiempo_Unix)
5
6   (let ((duracion_Ciclo 216)) ; tiempo del ciclo completo
7     (cond
8       ((and (>= (mod tiempo_Unix duracion_Ciclo) 0) (<= (mod
9         ((and (>= (mod tiempo_Unix duracion_Ciclo) 90) (<= (mo
10        ((and (>= (mod tiempo_Unix duracion_Ciclo) 210) (<= (m
11      )
12 )
13 )
```

On the right, the Windows PowerShell window shows the successful execution of the `sbcl` command, loading the file `core.lisp` without any errors:

```
PS C:\Users\Estudiante\Desktop\2do año\Paradigmas y lenguajes> sbcl
This is SBCL 2.6.5, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>

SBCL is free software, provided as is, with absolutely no warranty
It is mostly in the public domain; some portions are provided unde
r
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (load "core.lisp")
T
(temporizador 80)
ROJO
*
```

References

The Common Lisp - Dates and Times. (n.d.).

https://lispcookbook.github.io/cl-cookbook/dates_and_times.html

Scala. (n.d.). Youtube.

<https://www.youtube.com/watch?v=y-RnFbd8-V0&list=PLTd5ehIj0goOIRPRfwGGu9Bc2ap4bt7wR&index=5>

<https://www.scala-lang.org/>

<https://scastie.scala-lang.org/>

<https://docs.scala-lang.org/scala3/reference/enums/enums.html>

<https://docs.scala-lang.org/tour/pattern-matching.html>

<https://docs.scala-lang.org/tour/singleton-objects.html>

